Problem Set 5

Due February 20, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. [15 points] **State and Parameter Estimation:** The EKF can be used to estimate unknown parameters of a dynamical system, while also estimating its state. Consider the roll mode of a fixed-wing aircraft, with the model $\dot{p} = -ap + bu$, where p is roll rate, u is an aileron input, and a, b are unknown (fixed) model parameters. You receive a measurement $y(t) = p(t)$ from the IMU.

   (a) Formulate a discrete time approximation of this system $x_{t+1} = f(x_t, u_t) + w_t$, $y_t = g(x_t) + v_t$ with the state $x = [p \quad a \quad b]^T$.

   Solution: First discretize the continuous model using a 1st order Euler Approximation,

   $$\begin{aligned} p_{t+1} &= p_t + (-a_t p_t + b_t u_t)\delta t \\ a_{t+1} &= a_t \\ b_{t+1} &= b_t \\ y_t &= p_t \end{aligned}$$

   Therefore,

   $$x_{t+1} = f(x_t, u_t) + \omega_t = x_t + \begin{bmatrix} -\delta t a_t p_t + \delta t b_t u_t \\ 0 \\ 0 \end{bmatrix} + \omega_t$$

   $$y_t = g(x_t) + v_t = p_t + v_t$$

   where $x_t = \begin{bmatrix} p_t & a_t & b_t \end{bmatrix}^T$.

   (b) Find the Jacobians A_t and C_t for the EKF.

   Solution: Linearize the discrete model in problem (a):

   $$A_t = \frac{\partial f(x_t, u_t)}{\partial x_t} = \begin{bmatrix} 1 - \delta t a_t & -\delta t p_t & \delta t u_t \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

   $$C_t = \frac{\partial g(x_t)}{\partial x_t} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

   (c) Simulate the system with true parameters $a = 0.5$, $b = 2$, with an input $u_t = \sin(t)$, with $p(0) = 5$, and with additive process noise covariance $Q = \text{diag}([0.1 \quad 0 \quad 0])$ and measurement noise covariance $R = 0.1$. Implement an EKF to estimate the parameters a and b, as well as p. Use the noise covariances $Q = \text{diag}([0.1 \quad 0.01 \quad 0.01])$, $R = 0.1$ in your EKF. Start with $\Sigma_0 = 10I_3$ and $\mu_0 = 0$. Use $dt = 0.1s$, and simulate for 100 seconds.
   Remark: Think about what this result represents. You only have a model of your system, not an exact specification. But this method lets you estimate from measurements, in real-time, not just your real state but also the parameters of the system!

Solution:

Initial state for simulation: $p(0) = 5$.

Initial estimation of state: $x = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$, covariance matrix: $\Sigma = 10I_3$.

Time step: $dt = 0.1$, simulation time $= 100$.
See Fig 1, Fig 1 and Fig 3 for simulation results. Code solutions provided separately.

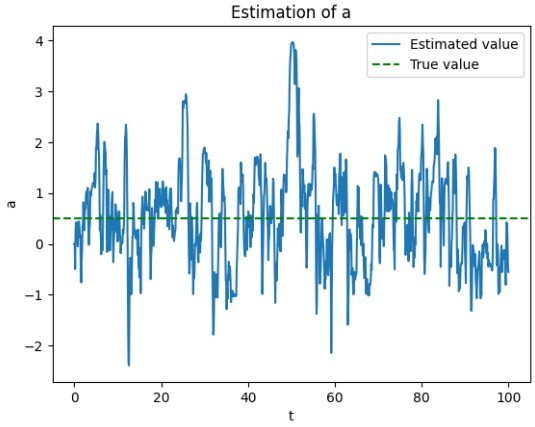


Figure 1: Plots of parameter estimation using EKF - parameter a

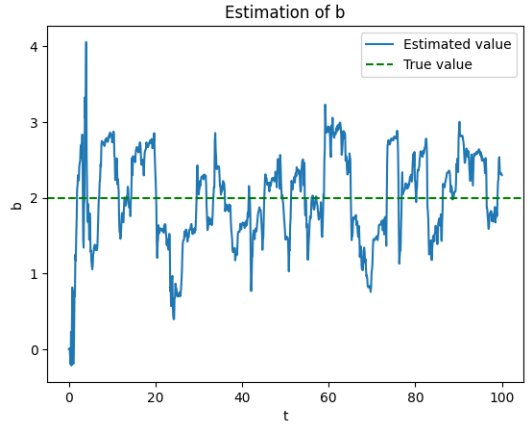


Figure 2: Plots of parameter estimation using EKF - parameter b

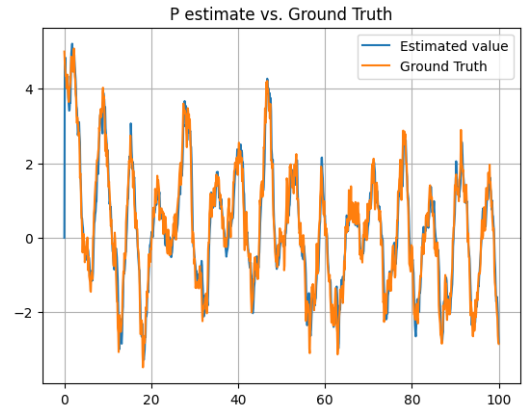


Figure 3: Plots of roll rate estimate using EKF

2. [15 points] **Kalman Filter as an Optimization Problem**: In class we claimed that the Kalman Filter update equations can be derived as the analytical solution to a least squares optimization problem. Let's prove the claim. Consider the Maximum A Posteriori (MAP) optimization problem we saw in class, $x_t^{\text{MAP}} = \arg\min_{x_t} J(x_t)$. With a linear measurement model $Y_t = C_t X_t + V_t$, $V_t \sim \mathcal{N}(0, R_t)$, and Gaussian prediction distribution $p(x_t \mid y_{1:t-1}) = \mathcal{N}(\mu_{t|t-1}, \Sigma_{t|t-1})$, we saw that the MAP objective function becomes a convex quadratic in x_t,

$$J(x_t) = \frac{1}{2}\|y_t - C_t x_t\|^2_{R_t^{-1}} + \frac{1}{2}\|x_t - \mu_{t|t-1}\|^2_{\Sigma^{-1}_{t|t-1}}.$$

   (a) Analytically find x_t^{MAP} by setting the gradient of the objective function to zero $\frac{\partial J(x_t)}{\partial x_t}\,|_{x_t = x_t^{\text{MAP}}} = 0$ and solving for x_t^{MAP}. Verify that $\mu_{t|t} = x_t^{\text{MAP}}$ by showing that your expression matches the mean update equation from the Kalman filter.

   (b) Analytically find the Hessian, $H = \frac{\partial^2 J(x_t)}{\partial x_t^2}\,|_{x_t = x_t^{\text{MAP}}}$. Show that $\Sigma_{t|t} = H^{-1}$ by verifying that the inverse of your expression for H matches the covariance update equation from the Kalman Filter.

   Hint: the matrix inverse lemmas that we have seen in previous problem sets will come in handy for this problem!

   (a) Analytically find x_t^{MAP} by setting the gradient of the objective function to zero $\frac{\partial J(x_t)}{\partial x_t}\,|_{x_t = x_t^{\text{MAP}}} = 0$ and solving for x_t^{MAP}. Show that your MAP estimate is $x_{t|t} = x_{t|t-1} + K_t(y_t - C_t\mu_{t|t-1})$ where $K_t = \Sigma_{t|t-1}C_t^T\left[C_t\Sigma_{t|t-1}C_t^T + R_t\right]^{-1}$. This is the update step of the Kalman filter.

   Solution: Since this is a quadratic objective, we can take the gradient of the objective with respect to x_t and set the expression to 0 and solve. First, we expand the objective

$$\begin{aligned} J(x_t) &= \frac{1}{2}(y_t - C_t x_t)^T R_t^{-1}(y_t - C_t x_t) + \frac{1}{2}(x_t - \mu_{t|t-1})^T \Sigma^{-1}_{t|t-1}(x_t - \mu_{t|t-1}) \\ &= \frac{1}{2}\left[y_t^T R_t^{-1} y_t - 2y_t^T R_t^{-1} C_t x_t + x_t^T C_t^T R_t^{-1} C_t x_t + \mu^T_{t|t-1}\Sigma^{-1}_{t|t-1}\mu_{t|t-1} - 2\mu^T_{t|t-1}\Sigma^{-1}_{t|t-1}x_t + x_t^T\Sigma^{-1}_{t|t-1}x_t\right]. \end{aligned}$$

   Then, we take the gradient and set to 0

$$\begin{aligned} \nabla_{x_t} J(x_t) &= -y_t^T R_t^{-1} C_t + x_t^T C_t^T R_t^{-1} C_t - \mu^T_{t|t-1}\Sigma^{-1}_{t|t-1} + x_t^T\Sigma^{-1}_{t|t-1} = 0 \\ &\downarrow \\ x_t^T C_t^T R_t^{-1} C_t + x_t^T\Sigma^{-1}_{t|t-1} &= y_t^T R_t^{-1} C_t + \mu^T_{t|t-1}\Sigma^{-1}_{t|t-1} \\ &\downarrow \\ (C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1})x_t &= C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1} \\ &\downarrow \\ x_t &= (C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1})^{-1}(C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1}). \end{aligned}$$

   We apply the Woodbury matrix identity $(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$ to the first term to yield

$$\begin{aligned} x_t &= (\Sigma_{t|t-1} - \Sigma_{t|t-1}C_t^T \underbrace{(R_t + \overbrace{C_t\Sigma_{t|t-1}C_t^T}^{B_t})^{-1}}_{S_t^{-1}} C_t\Sigma_{t|t-1})(C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1}) \\ &= \mu_{t|t-1} + \underbrace{\Sigma_{t|t-1}C_t^T R_t^{-1} y_t - \Sigma_{t|t-1}C_t^T S_t^{-1} C_t\Sigma_{t|t-1}C_t^T R_t^{-1} y_t} - \Sigma_{t|t-1}C_t^T S_t^{-1} C_t\mu_{t|t-1}. \end{aligned}$$

   Let's simply the middle two terms. We can rewrite the middle two terms as

$$\Sigma_{t|t-1}C_t^T R_t^{-1} y_t - \Sigma_{t|t-1}C_t^T S_t^{-1} C_t\Sigma_{t|t-1}C_t^T R_t^{-1} y_t = \Sigma_{t|t-1}C_t^T\left[I - S_t^{-1} C_t\Sigma_{t|t-1}C_t^T\right] R_t^{-1} y_t.$$

Let's focus our attention to the bracketed term. Note that S_t can be expressed as $S_t = R_t + B_t$, therefore the bracketed term is $I - S_t^{-1}(S_t - R_t) = S_t^{-1}R_t$. Finally, we have the following expression

$$\begin{aligned} x_t &= \mu_{t|t-1} + \Sigma_{t|t-1}C_t^T \left[I - S_t^{-1}C_t\Sigma_{t|t-1}C_t^T\right] R_t^{-1}y_t - \Sigma_{t|t-1}C_t^T S_t^{-1}C_t\mu_{t|t-1} \\ &= \mu_{t|t-1} + \Sigma_{t|t-1}C_t^T S_t^{-1}R_t R_t^{-1}y_t - \Sigma_{t|t-1}C_t^T S_t^{-1}C_t\mu_{t|t-1} \\ &= \mu_{t|t-1} + \Sigma_{t|t-1}C_t^T S_t^{-1}y_t - \Sigma_{t|t-1}C_t^T S_t^{-1}C_t\mu_{t|t-1} \\ &= \mu_{t|t-1} + \Sigma_{t|t-1}C_t^T S_t^{-1}(y_t - C_t\mu_{t|t-1}). \end{aligned}$$

(b) Analytically find the Hessian, $H = \frac{\partial^2 J(x_t)}{\partial x_t^2}\,|_{x_t = x_t^{\text{MAP}}}$. Show that the Hessian has the form $H = C_t^T R^{-1} C_t + \Sigma_{t|t-1}^{-1}$.

Solution: We take the gradient once more.

$$\begin{aligned} \nabla_{x_t} J(x_t) &= (C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1})x_t - C_t^T R_t^{-1} y_t + \Sigma_{t|t-1}^{-1}\mu_{t|t-1} \\ \nabla_{x_t}^2 J(x_t) &= H_t = C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1} \end{aligned}$$

(c) Invert H and simplify to get $\Sigma_{t|t} = H^{-1} = \Sigma_{t|t-1} - KC_t\Sigma_{t|t-1}$. This is the covariance update for the Kalman filter.

Solution: We again apply the Woodbury identity to part (b)

$$\Sigma_{t|t} = (C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1})^{-1} = \Sigma_{t|t-1} - \underbrace{\Sigma_{t|t-1}C_t^T(R_t + C_t\Sigma_{t|t-1})^{-1}}_{K_t} C_t\Sigma_{t|t-1}.$$

3. [20 points] **EKF SLAM**: Consider again the differential drive robot model,

$$\begin{aligned} P_{t+1}^x &= P_t^x + \delta t\, s_t \cos\Theta_t + W_t^x \\ P_{t+1}^y &= P_t^y + \delta t\, s_t \sin\Theta_t + W_t^y \\ \Theta_{t+1} &= \Theta_t + \delta t\, \phi_t + W_t^\theta \end{aligned}$$

where $P_t = [P_t^x \quad P_t^y]^T$ is the robot position relative to a feature point, Θ_t is its heading angle, s_t its speed input, ϕ_t its rotation rate input, and W_t is the process noise distributed as $W_t \sim \mathcal{N}(0, Q)$ with $Q = 0.1\delta t I_3$. Suppose you have four features at positions $m_1 = [0 \quad 0]^T$, $m_2 = [10 \quad 0]^T$, $m_3 = [10 \quad 10]^T$, and $m_4 = [0 \quad 10]^T$. Now suppose the robot takes relative position measurements to each of the features (in the robot's local frame), with measurement model,

$$Y_t^j = \begin{bmatrix} \cos(\Theta_t) & \sin(\Theta_t) \\ -\sin(\Theta_t) & \cos(\Theta_t) \end{bmatrix} (m_j - P_t) + V_t^j,$$

as we saw in class. The measurements have an associated measurement uncertainty of $V_t^j \sim \mathcal{N}(0, R)$, where $R = 0.25 I_2$. Also suppose that the robot pose is initially distributed $\mathcal{N}([1m\ 1m\ 0rad]^T, I_3)$ is commanded with a known control sequence $s_t = 0.5$ and $\phi_t = \sin(2t)\text{rad/s}$. You can assume that the ground truth trajectory also starts at $[1m\ 1m\ 0rad]^T$. Use $\delta t = 0.5s$ and run for 100 timesteps (i.e., $50s$).

We highly recommend using the starter code from Problem Set 4 Q4, and editing it as necessary for this problem.

(a) Calculate the measurement Jacobian C_t for the measurement model.

Solution: We calculate the measurement Jacobian by taking the Jacobial with respect to the state

$$x = \begin{bmatrix} P^x \\ P^y \\ \Theta \end{bmatrix} \tag{1}$$

Based on the measurement model provided above, we get the Jacobian for a single feature to be

$$C_t^j = \begin{bmatrix} -\cos(\Theta_t) & -\sin(\Theta_t) & -\sin(\Theta_t)\left(m_j^x - P_t^x\right) + \cos(\Theta_t)\left(m_j^y - P_t^y\right) \\ \sin(\Theta_t) & -\cos(\Theta_t) & -\cos(\Theta_t)\left(m_j^x - P_t^x\right) - \sin(\Theta_t)\left(m_j^y - P_t^y\right) \end{bmatrix} \tag{2}$$

The complete measurement Jacobian for all four features is the stack of the matrix above for each feature.

$$C_t = \begin{bmatrix} C_t^1 \\ C_t^2 \\ C_t^3 \\ C_t^4 \end{bmatrix} \tag{3}$$

(b) Implement an EKF for the robot to localize itself with these measurements given known feature locations.

Solution: The dynamics Jacobian is the same as was used in HW4.

$$A_t = \begin{bmatrix} 1 & 0 & -\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & \delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix} \tag{4}$$

$$\tag{5}$$

With our dynamics model and measurement model complete, we can implement the EKF. The resultant plots are as follows.

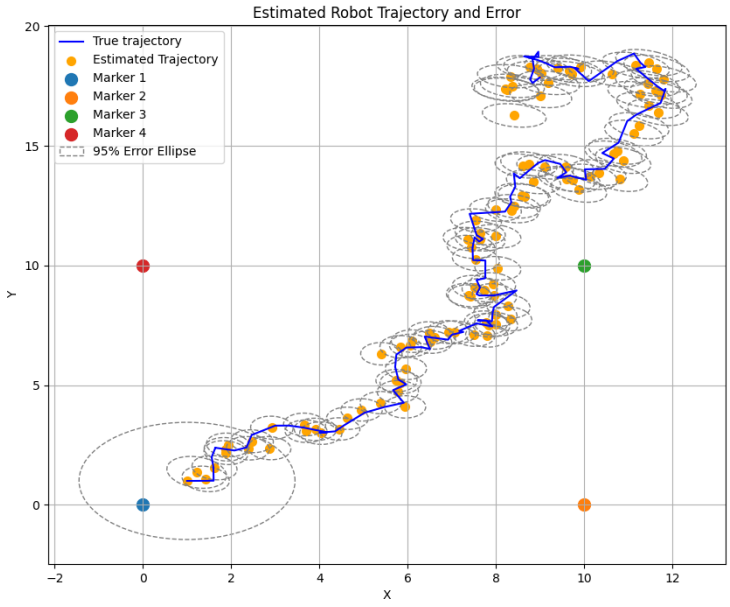


Figure 4: EKF robot position estimate

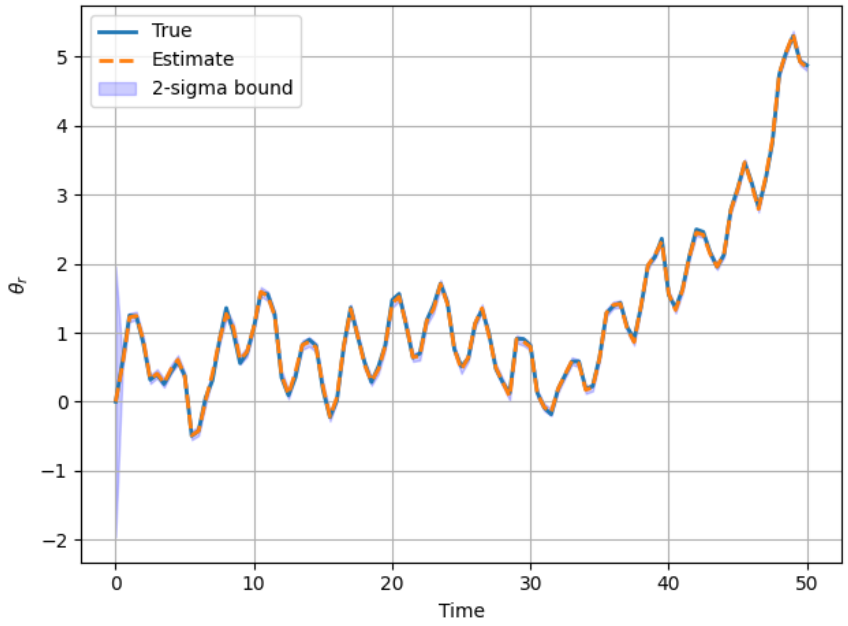


Figure 5: EKF robot heading estimate

(c) Implement an EKF to map the features given known robot state trajectory. Use a *noiseless* ground-truth trajectory created by the given inputs as your reference. Start your initial mean positions at the origin, with a large covariance in the position ($\Sigma_0 = 10^5 I$) For each of parts (b), (c), and (d), provide a plot with

- the mean estimate of the state (whether that is the robot's location, the feature positions, or both),
- the true trajectory and true positions of the features,
- the covariance ellipses around the mean position estimates.
- On a separate plot, include the Θ time series estimate, the true Θ trajectory, and the 95% confidence bounds around the estimate.

Solution: We first generate a true trajectory based on the control inputs and with no noise. Here, our state is the positions of the features:

$$x = \begin{bmatrix} m_1^x \\ m_1^y \\ m_2^x \\ m_2^y \\ m_3^x \\ m_3^y \\ m_4^x \\ m_4^y \end{bmatrix} \tag{6}$$

Since the features don't move, our dynamics model and A matrix are identity I_8. The measurement

Jacobian is as follows

$$C_t = \begin{bmatrix} \cos(\Theta_t) & \sin(\Theta_t) & 0 & 0 & 0 & 0 & 0 & 0 \\ -\sin(\Theta_t) & \cos(\Theta_t) & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \cos(\Theta_t) & \sin(\Theta_t) & 0 & 0 & 0 & 0 \\ 0 & 0 & -\sin(\Theta_t) & \cos(\Theta_t) & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \cos(\Theta_t) & \sin(\Theta_t) & 0 & 0 \\ 0 & 0 & 0 & 0 & -\sin(\Theta_t) & \cos(\Theta_t) & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cos(\Theta_t) & \sin(\Theta_t) \\ 0 & 0 & 0 & 0 & 0 & 0 & -\sin(\Theta_t) & \cos(\Theta_t) \end{bmatrix} \tag{7}$$

The true trajectory, the true positions of the markers, the positions of the markers over time, and the covariance matrices are provided in the Figures below. However, we still have the measurement noise V_t as before. For this problem, we start with a large initial covariance since we are very unertain of the feature's positions. Since we also know that the features do not move with time, we can set the $Q \approx 0$. Specifically, I set $Q = 10^{-5} I_8$.

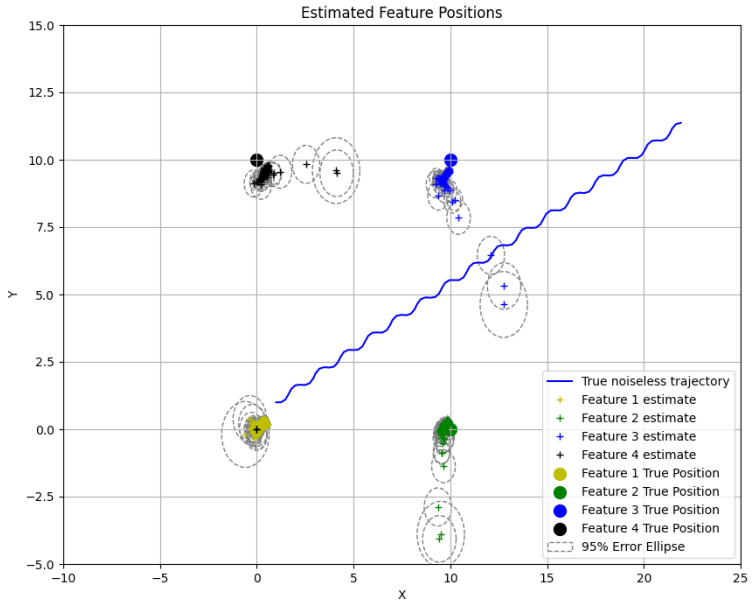


Figure 6: EKF feature estimates

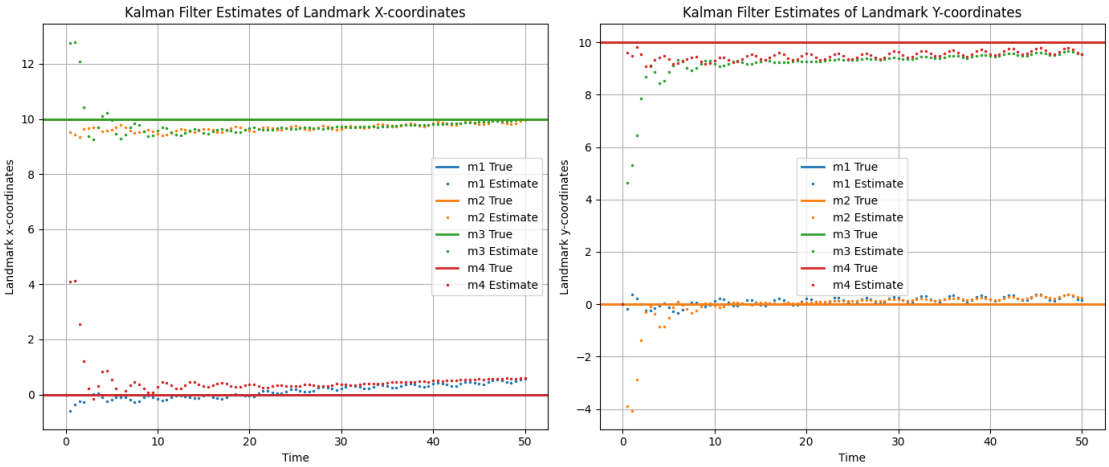


Figure 7: EKF estimate of features over time

(d) Combine the two items above to jointly estimate the robot pose and feature locations. This is EKF SLAM, the original SLAM solution.

Solution: Now, our state is the concatenation of both the robot state and the features. So, we will simultaneously be estimating the robot's state and the feature locations. This is a very rudimentary form of SLAM, called EKF Slam.

$$x = \begin{bmatrix} P^x \\ P^y \\ \Theta \\ m_1^x \\ m_1^y \\ m_2^x \\ m_2^y \\ m_3^x \\ m_3^y \\ m_4^x \\ m_4^y \end{bmatrix} \tag{8}$$

The A and C matrices also get concatenated accordingly, with A_t having a size 11×11, and C having a size 8×11. We start with identity covariance on the robot state, but a large covariance 10^5 on the feature locations as in part (c). This allows for better convergence to the true locations of the features. The results from running EKF slam are shown below.

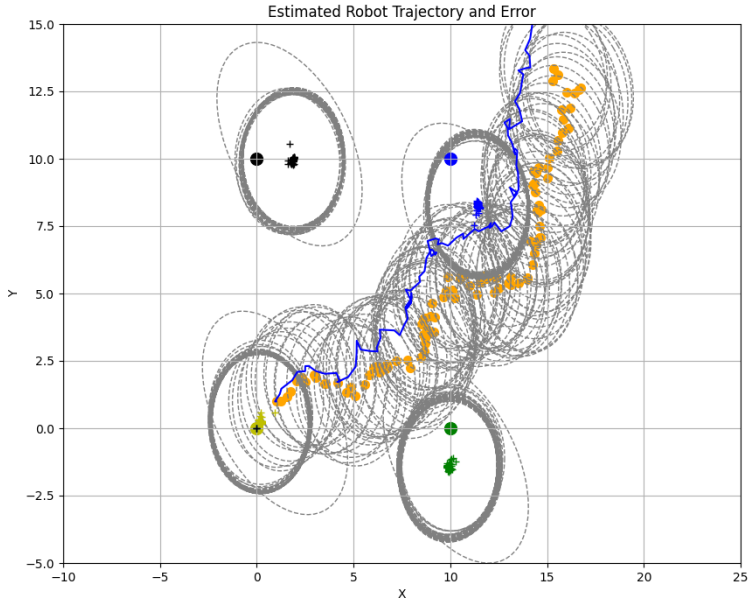


Figure 8: EKF robot and feature estimates

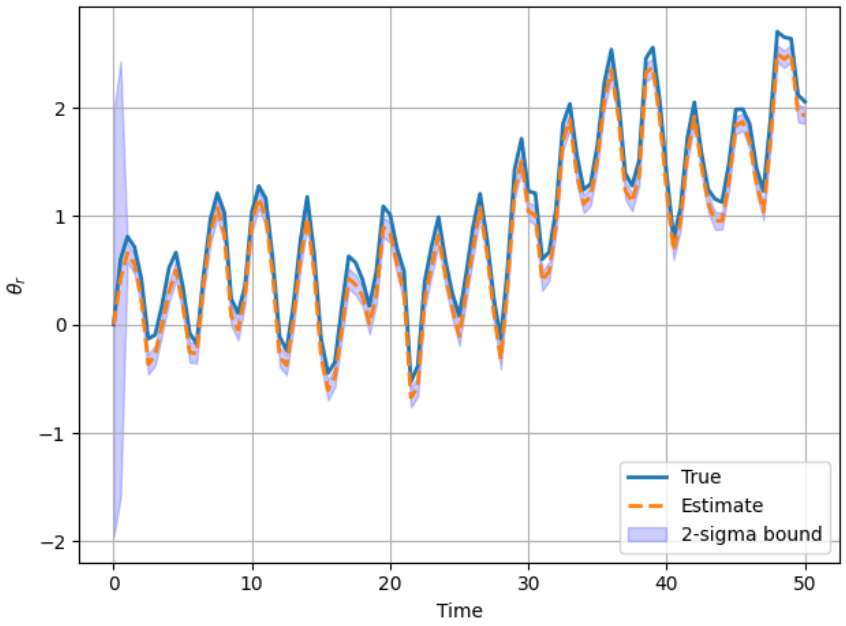


Figure 9: EKF robot heading estimate

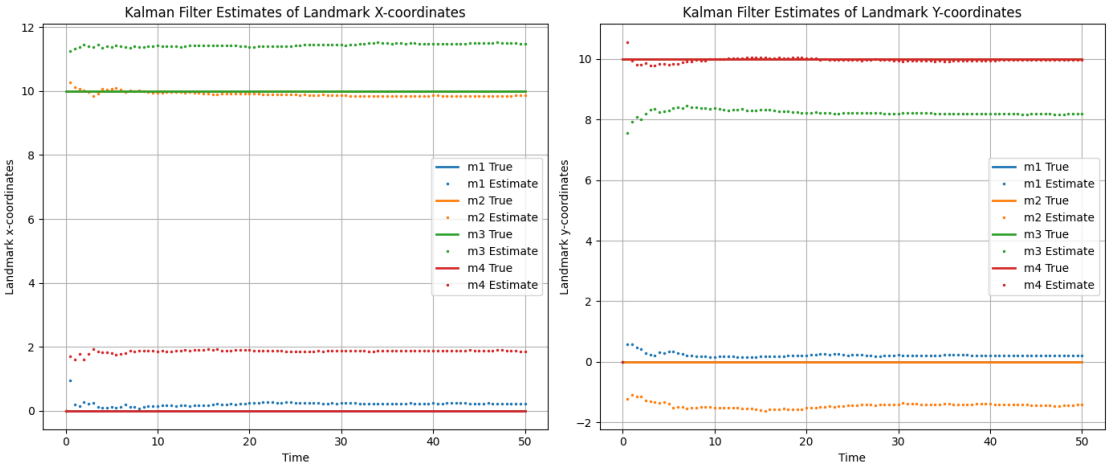


Figure 10: EKF estimate of features over time

Clearly, the results of EKF slam are poor. With some tuning and better initial guesses of the feature positions, the results can be improved, but this method is limited to small-dimensional problems. You can see how with an increase in the number of features, your state vector quickly becomes very large, and implementing the EKF becomes computationally expensive.